

The background is a blurred image of a computer monitor. On the left, a portion of a Windows calculator is visible, showing a grid of buttons. The main area of the screen displays lines of code in a dark-themed editor, with some text highlighted in green. At the bottom, a Windows taskbar is visible with various application icons.

C#

Level 1

Sum of Matrix



C#

Level 1

Sum of Matrix

Definition

Nullable Type

-  A nullable type can store a normal value or `null`.
-  It is mainly used with value types that normally cannot contain `null`.

Explanation

Nullable<int>

-  `Nullable<int>` can store any `int` value or `null`.

-  It can be declared using `Nullable<T>` syntax.

Procedures

1. ☐ Declare a nullable value type.
2. ☐ Assign a normal value or ``null``.
3. ☐ Use ``HasValue`` to check whether a value exists.
4. ☐ Access the value only when ``HasValue`` is ``true``.

💡 Example

```
Nullable<int> i = null;  
Console.WriteLine(i.HasValue);
```

➡ Result
False

Note

- ◆ `int` normally cannot store `null`.
 `int i = null;` ❌
- ◆ `Nullable<int>` can store `null`.
 `Nullable<int> i = null;` ✔️
- ◆ Nullable types can also use the `?` shorthand:
int? i = null;

⊘ Warning

- ◆ Do not access the value of a nullable variable when it is `null`.



Comparison

▨ `int`

- ✓ Stores integer values.
- ✓ Cannot normally store `null`.

▤ `int?` / `Nullable<int>`

- ✓ Stores integer values or `null`.
- ✓ Can be used when a value may be missing.

✦ Summary

- ✓ Nullable types can store a value or `null`.
- ✓ `Nullable<int>` and `int?` represent the same nullable integer type.
- ✓ `HasValue` checks whether a value exists.
- ✓ `true` → value exists.
- ✓ `false` → value is `null`.

Thank you 😊